[CH2] 프롬프트 엔지니어링 / 데이터 수집 & API 서버 구축 / LangChain 과제 발제

📌 Lv2에서 학습한 내용을 직접 적용해 보는 과제입니다.

여러분은 KANT NEXT의 신입 LLM 엔지니어 지원자로서 **DevDesk 개발 요청을 읽고 기능을 구현**합니다.

이번 과제는 **프롬프트 엔지니어링 / 데이터 수집 & API 서버 구축** / LangChain 세 가지입니다. 각 과제의 구현 파일(`main.py` 또는 `app.py`)과 `AI_USAGE.md` 두 파일만 안내된 과제 제출 시스템에 제출합니다.

제출 일정

항목	일정
제출 시작	10/13일 (화)
제출 마감	10/27일 (화)

- 제출 경로는 과제 운영 공지를 확인하세요.

진행 안내

진행 방식

1. KANT NEXT DevDesk 과제 웹사이트에서 회사 소개와 과제 전형 안내를 읽습니다.
2. 과제 목록에서 해당 과목을 선택하고, **시작 가이드 → 과제 안내 → AI 활용 기준 → 제공 자료 안내**를 확인합니다.
3. 과목별 **학생용 ZIP**을 내려받아 압축을 풀고, VS Code에서 과목별 구현 파일(`main.py` 또는 `app.py`)과 `pyproject.toml` 이 있는 폴더를 엽니다. `uv` 와 Python 3.12로 환경을 준비합니다.
4. 제공된 코드의 TODO를 한 기능씩 완성하고 **기본 실행 → 결과 확인 → 기능 검사 → 필요한 실제 연결 확인** 순서로 진행합니다. 고정 응답을 사용하는 **fixture 실행**과 **실제 API 또는 모델 실행**을 구분해 실행합니다.
5. 아래 과목별 제출 목록과 웹사이트 제출 안내를 확인한 뒤, 운영 공지로 안내받은 **과제 제출 시스템**에 제출합니다.

🌐 **어디서 무엇을 확인하나요?**

- **이 발제 페이지:** 공통 진행 안내, 제출 일정, 세 과제의 요구사항을 확인합니다.
- **과제 웹사이트:** 회사의 개발 요청을 읽고 시작 가이드, 과제 본문, 학생용 ZIP을 확인합니다.

코드는 내려받은 프로젝트에서 작성하고 실행합니다. 완성한 파일은 운영 공지의 과제 제출 시스템에 제출하세요.

- **프롬프트 엔지니어링:** 정책과 답변 검증, 도구, 이미지 입력, 재시도와 사용량 기록을 구현합니다. 필수 3문제와 선택 심화 1문제입니다.
- **데이터 수집 & API 서버 구축:** 공개 자료를 수집하고 정제한 뒤 문서 API와 질문 API를 구현합니다. 필수 4문제와 선택 심화 1문제입니다.
- LangChain: 검색, 체인, 대화 이력, 오류 복구, 스트리밍을 연결합니다. 필수 5문제와 선택 심화 1문제입니다.

세 과제는 같은 DevDesk 프로젝트를 다루며, **이전 과제의 완성 코드가 없어도 제공 자료로 시작**할 수 있습니다. 선택 심화는 필수 구현을 마친 뒤 도전하세요.

명령어는 이렇게 읽으세요.

- `uv run --frozen python main.py` 또는 `uv run --frozen python app.py` : 제공된 잠금 파일을 갱신하지 않고 과제 환경의 Python으로 구현 파일을 실행합니다. ZIP을 푼 프로젝트 폴더에서 입력하세요.
- `demo` : 제공 예제로 설치 상태를 확인합니다. 작성한 코드의 완료를 뜻하지 않습니다.
- `run 기능명` : 작성한 기능을 제공 입력으로 실행합니다. 아래 과목별 표의 결과 파일을 열어 확인하세요.
- `check 기능명` : 정상 입력과 오류 입력으로 요구사항을 검사합니다. 실패한 항목의 이름과 이유를 보고 해당 함수를 수정합니다. `check all` 은 필수 전체 검사이며 선택 심화는 별도입니다.
- `--live` : 지원하는 실행 명령에 붙여 실제 외부 API를 사용합니다. `check` 에는 붙이지 않습니다. 실제 호출이 필요한 기능과 준비 사항은 과목별 안내를 따르세요.

`support.py` 는 실행, 데이터 읽기와 저장을 돕고 `checks.py` 는 작성한 코드를 검사합니다. 두 파일의 내부를 모두 이해할 필요는 없으며 제공된 상태로 사용합니다. `data/` 는 제공 입력이고 `artifacts/` 는 실행 후 생성되는 결과입니다.

[실행 순서]

기능 구현 → `run` 실행 → 결과 파일 확인 → `check` 검사 → 필요한 기능의 `--live` 실행

- `run` 이 실행됐다고 구현이 모두 올바른 것은 아닙니다. 결과 파일을 읽고 `check` 로 정상·오류 상황을 함께 검사하세요.
- `check` 를 통과했다고 실제 모델 답변까지 정확한 것은 아닙니다. 실제 답변은 제공 문서의 내용과 직접 대조하세요.
- `--live` 는 안내된 명령에만 붙입니다. `check` 에는 붙이지 않습니다. 서버처럼 `run` 대신 `serve` 로 실행하는 기능도 있습니다.
- `--live` 가 항상 모델 호출을 뜻하지는 않습니다. `collect --live` 는 GitHub 호출, `query --live` 는 모델 호출입니다. `documents --live` 는 이미 수집한 실제 데이터 파일을 정제하며 외부에 접속하지 않습니다.

제출 시 유의사항

- **구현 파일과 AI 사용 기록 두 개만 제출하세요.** 과목별 실행과 `check all` 을 마친 뒤 결과를 확인합니다. `artifacts/` 는 본인 확인용이며 제출하지 않습니다.
- **실제 호출이 필요한 항목을 확인하세요.** fixture 통과만으로 실제 모델의 답변 품질, 도구 선택, 캐시 적중을 검증했다고 기록하지 않습니다. 실행하지 못했다면 성공 결과를 만들지 말고 실패 상황과 확인한 범위를 남기세요.
- **코드의 짧은 설명 주석과 `AI_USAGE.md` 를 작성하세요.** 과목별로 요구한 이유, 관측 메모를 해당 코드 위치에 남깁니다. 별도 보고서나 이론 답안은 제출하지 않습니다.
- **배포된 파일과 검사 기준을 유지하세요.** `support.py`, `checks.py`, 고정 데이터와 검사를 바꿔 통과시키지 마세요. 직접 작성할 파일과 허용된 변경 범위는 각 과목 안내를 따릅니다.
- **비밀값과 로컬 환경을 제외하세요.** `.env`, 실제 API 키, 인증 헤더, `.venv/`, `__pycache__/` 는 제출물과 AI 대화에 포함하지 않습니다. 제공 코드, `data/`, `artifacts/` 와 환경 설정 파일은 제출하지 않습니다.

수강생 과제 수행 TIP!

1. 튜터님께 질문할 때는 현재 상황과 시도한 내용을 함께 전달해, 스스로 해결 방향을 찾을 수 있도록 질문하세요.

   [4단계 질문 프레임워크]

   1. 현재 상황 / 하고자 하는 목표

      어떤 과제의 어느 기능을 구현하고 있으며, 기대하는 동작은 무엇인지 설명해주세요.

   2. 시도한 방법 / 현재 진행 상태

      직접 작성한 내용과 실행한 명령, 이미 확인한 사항을 정리해주세요.

   3. 어려운 점 / 막힌 지점

      예상 결과와 실제 결과가 어떻게 다른지, 어느 검사에서 막혔는지 설명해주세요.

   4. 문제 관련 참고 자료 / 관련 코드

      필요한 코드와 오류 메시지, 참고 자료를 공유하되 API 키와 개인정보는 제거해주세요.

2. **AI를 활용하더라도 구현과 검증은 직접 하세요.** 개념을 이해하거나 직접 작성한 코드의 오류 원인을 찾는 데 활용할 수 있습니다.

   필수와 선택 심화에 같은 AI 활용 기준을 적용합니다. 개념 설명과 자기 코드의 오류 원인, 확인 방법을 질문할 수 있습니다. **과제 전체, 미작성 TODO, 프롬프트 문장 또는 심화 설계를 대신 완성해 달라고 요청하지 않습니다.**

   - 좋은 예: HTTP 오류 코드의 차이 질문하기 / 직접 작성한 조건문과 오류를 제시하고 원인 설명 요청하기
   - 지양할 예: 문제를 붙여 넣고 완성 코드 받기 / baseline, reasoning 프롬프트 대필 요청하기 / 실행하지 않은 결과, 비용, 캐시 수치 만들기
   - AI의 설명을 이해한 뒤 직접 수정하고, 관련 검사를 다시 실행하세요. 완성 풀이가 제공되면 개념, 원인 설명으로 질문 범위를 좁히세요.
   - `AI_USAGE.md` 의 과목별 양식에 사용 도구, 질문, 이해한 내용, 직접 수정한 부분, 재실행 결과를 사실대로 남깁니다. 사용하지 않았다면 **사용하지 않음**이라고 적습니다.
   - **제출한 코드의 입력, 분기, 반환값과 프롬프트의 선택 이유를 본인 말로 설명할 수 있어야 합니다.**

세부 과제 소개

▸ 프롬프트 엔지니어링 과제

▾ 데이터 수집 & API 서버 구축 과제

📝 과제 소개

공개 릴리스 자료를 수집하고 정제한 뒤, 다른 프로그램이 문서를 등록하거나 조회하고 LLM에 질문할 수 있는 **DevDesk API 서버**를 구현합니다. **필수 4문제와 선택 심화 1문제**이며, `app.py` 의 제공 코드에서 필요한 조건과 값을 직접 작성합니다.

🧑‍💻 실무 요청서

> **KANT NEXT 개발지원팀의 요청**
> DevDesk가 공개 릴리스 자료를 근거로 답하도록 개선하려고 합니다. GitHub의 `astral-sh/uv` 릴리스를 모아 믿고 사용할 문서로 정리하고, 다른 서비스가 문서를 조회하거나 질문할 수 있는 API를 만들어 주세요. 정상 응답뿐 아니라 잘못된 입력, 외부 모델의 실패, 과도한 요청도 처리할 수 있어야 합니다.

🚩 과제 목표

- 공개 API의 페이지와 오류를 확인하며 원천 자료와 수집 결과를 다시 읽을 수 있게 저장합니다.
- 결측, 형식 오류, 중복을 점검하고 본문, 출처, 문서 ID, 수집시각을 가진 문서를 구성합니다.
- FastAPI와 Pydantic으로 문서, 질문 API의 요청과 응답 규격을 연결합니다.
- 선택한 문서로 LLM 답변을 만들고, 출처 검증, 재시도, 요청 제한, 안전한 기록을 적용합니다.

📁 과제 파일 링크

- 과제 본문, 시작 안내, 제공 자료 설명, AI 사용 안내
- 학생용 ZIP 다운로드 — kant-next-api-student.zip

💻 실행 환경

VS Code, uv, Python 3.12를 사용합니다. ZIP을 풀고 `app.py` 와 `pyproject.toml` 이 있는 폴더를 VS Code로 연 뒤 같은 위치에서 터미널을 실행하세요. uv가 없다면 공식 설치 안내를 따릅니다.

```
uv --version
uv sync --frozen
uv run --frozen python --version
uv run --frozen python app.py demo
```

`uv sync --frozen` 은 제공된 `pyproject.toml` 과 `uv.lock` 을 사용해 `.venv` 에 필요한 패키지를 설치합니다. `.python-version` 이 Python 3.12를 선택합니다. `uv run --frozen python --version` 의 결과에 Python 3.12가 표시되는지 확인하세요.

이후 실행과 검사는 프로젝트 폴더에서 안내된 `uv run --frozen python ...` 명령을 사용합니다. 가상환경을 별도로 활성화할 필요는 없습니다.

`demo` 는 키 없이 예시를 보여 주는 환경 확인 명령이며 아직 작성하지 않은 기능은 실행하지 않습니다.

기본 **fixture**는 고정 데이터와 모델 대역을 사용합니다. `--live` **를 붙인 실행만 실제 자료나 모델을 사용**하며, 결과는 `artifacts/fixture/` 와 `artifacts/live/` 로 나뉩니다. 검사 명령은 실제 키, 별도 서버, 외부 호출 없이 필요한 자료와 테스트 서버를 준비합니다.

실제 모델 실행 전 Windows는 `Copy-Item .env.example .env`, macOS/Linux는 `cp .env.example .env` 로 복사하고 `.env` 의 `OPENAI_API_KEY` 에 받은 키를 넣습니다. 제공 기본 모델은 `gpt-5.6-luna` 이며 계정에서 이용할 수 있는 모델로 설정하세요. `.env` 값을 출력하거나 제출하지 않습니다.

📊 제공 데이터

- `data/documents.json` : 기존 여섯 필드와 ID를 유지한 회사 정책 6개.
- `data/release_pages.json` : 수집과 정제 검사용 고정 HTTP 응답. 2페이지에 각 5행이며 실제 릴리스, 배포 사실을 뜻하지 않습니다.
- `data/query_example.json` : `policy-python` 문서 1개를 선택하는 질문 요청 예시.
- `data/snapshot_before.json`, `data/snapshot_after.json` : 선택 심화용으로 같은 범위를 비교하는 전체 스냅샷.

정책의 `.invalid` 주소는 테스트용 출처 표시이며 접속 대상이 아닙니다. 실제 수집 대상은 GitHub 공개 릴리스 API입니다. 고정 자료의 건수와 ID를 실제 수집 결과에 그대로 적용하지 마세요.

파일별 역할

- `app.py` : 직접 구현할 코드. `support.todo(...)` 를 주석에 맞는 조건, 값, 코드로 바꿉니다. `None` 으로 바꾸거나 지우는 것만으로 구현이 완성되지 않습니다.
- `support.py` : 실행 명령, 서버 연결, 문법 처리와 테스트 응답. `checks.py` : 기능별 검사와 실패 입력. 검사 통과를 위해 기대값이나 제공 데이터를 바꾸지 않습니다.
- `README.md`, `ASSIGNMENT.md` : 실행, 업무 요청 안내. `AI_USAGE.md` : 직접 작성할 AI 사용 기록. `artifacts/` : 실행 결과.

반복문, 파일 저장, 오류 처리와 비동기 실행 순서, HTML 정리, URL, 날짜 문법 검사, 데코레이터와 서버 시작 코드는 제공됩니다. 수집, 정제, 조회, 재시도 함수의 `작성 메모` 에는 자신의 구현 이유를 각각 **1~2줄**로 남기세요. API 선택, 본문과 메타데이터 구분, HTTP 오류의 차이, 비동기 대기와 재시도 상한을 자기 코드와 연결해 설명합니다.

실행 결과 확인

`data/` 에는 정책 문서 6개, 수집을 재현하는 릴리스 10행, 질문 요청 예시와 선택 심화용 스냅샷 두 개가 있습니다. 제공 입력의 내용은 예제 자료 안내에서 확인합니다. `artifacts/` 는 실행 후 생성되는 결과 폴더입니다.

자료 수집에서는 다음 세 명령을 구분하세요.

전체 명령	하는 일	확인할 곳
`uv run --frozen python app.py run collect`	제공된 릴리스 자료로 작성한 수집 함수를 실행합니다. 외부에 접속하지 않습니다.	`artifacts/fixture/collection_report.json` 의 `success=true`, `successful_pages=2`, `raw_count=10` 과 `raw_pages.json` 의 두 페이지
`uv run --frozen python app.py check collect`	정상 수집, 오류 응답과 페이지 상한 등을 준비된 입력으로 검사합니다. 별도의 서버나 키가 필요하지 않습니다.	터미널의 PASS/FAIL과 `artifacts/fixture/checks-collect.json` 의 `failed=0`
`uv run --frozen python app.py run collect --live`	GitHub 공개 API에서 현재 릴리스를 최대 두 페이지 수집합니다. 인터넷 연결이 필요하며 OpenAI 키는 사용하지 않습니다.	`artifacts/live/` 의 같은 두 수집 파일. 성공 여부와 건수를 확인하되 fixture의 10행과 같다고 가정하지 않습니다.

수집 구현과 fixture 검사를 마친 뒤 실제 수집도 확인합니다. `check collect` 가 만든 검사 결과는 다음 정제 단계의 입력을 대신하지 않습니다. 정제 전에 반드시 같은 모드의 `run collect` 를 실행하세요.

아래 명령 부분 앞에는 `uv run --frozen python app.py` 를 붙입니다. 기본 실행 파일은 `artifacts/fixture/` 에 생성됩니다.

명령 부분	생성되는 파일 또는 화면	확인할 내용
`run documents`	`documents.json`, `quarantine.json`, `excluded.json`, `quality_report.json`	원본 10행이 채택 3개, 격리 5개, 제외 2개로 나뉘는지 봅니다. `quality_report.json` 의 건수와 사유를 대조합니다.
`run query`	`model_response.json`	`response` 의 답변과 출처가 선택 문서에 맞는지 확인합니다. `--live` 실행은 같은 파일을 `artifacts/live/` 에 저장합니다.
`run retry`	`checks-retry.json`	고정 실패 검사입니다. `check retry` 와 같은 검사를 실행하며 `failed=0` 인지 확인합니다.
`check all`	`checks-all.json`	필수 검사 전체의 `failed=0` 을 확인합니다. 실패하면 `results` 의 `name` 과 `detail` 을 읽고 해당 기능을 다시 검사합니다.
`serve --stage document-api` / `serve`	브라우저 `/docs` 의 응답과 서버 터미널	상태 코드와 응답 본문, 요청 제한과 안전한 로그를 확인합니다. 응답이나 서버 로그 파일은 자동 생성되지 않으며 캡처 제출도 필요하지 않습니다.
`challenge`	`document_changes.json`	선택 심화의 added, changed, unchanged, missing 분류를 확인합니다.

`run documents --live` 는 외부에 접속하지 않고 직전에 수집한 live 파일을 정제합니다. 수집을 다시 하면 같은 모드로 정제도 다시 해야 합니다. 같은 이름의 결과 파일은 재실행하면 바뀌므로 비교가 필요할 때만 따로 복사해 둡니다. 결과 파일은 확인용이며 제출하지 않습니다.

🔝 제출 형식

과제 제출 시스템에는 **`app.py` 와 `AI_USAGE.md` 두 파일만 제출**합니다.

1. `app.py` : 구현 코드와 지정된 위치의 짧은 이유 또는 관측 주석
2. `AI_USAGE.md` : AI 사용 기록. 사용하지 않았다면 `사용하지 않음` 으로 기록

`support.py`, `checks.py`, `data/`, `artifacts/` 와 환경 설정 파일은 제출하지 않습니다. 실행 결과 확인은 과제 수행에 포함되지만 파일이나 화면 캡처를 제출할 필요는 없습니다. `.env`, `.venv`, 캐시, 실제 키와 개인 정보도 제외합니다.

아래 문제에서 설명하는 결과 파일은 본인이 동작을 확인하기 위한 자료입니다. 별도 보고서나 이론 답안은 작성하지 않습니다.

- AI는 **개념 질문과 직접 작성한 코드의 디버깅**에만 사용합니다. 전체 풀이, TODO 완성 코드, 참고 답안은 요청하지 않습니다. `AI_USAGE.md` 에는 사용 여부, 도구와 도움마다 단계/함수, 직접 시도한 내용, 질문 요약, 받은 설명, 직접 수정, 확인한 내용, 재실행 결과를 적습니다. 미사용 시 `사용하지 않음` 으로 남깁니다. 키, `.env`, 인증 헤더를 AI 질문에 넣지 않습니다.

⏱ 권장 시간과 난이도

필수 1부터 **정상 입력 → 오류 입력 → 기능 검사** 순서로 진행하세요. 필수 문제에서는 제공된 반복문과 서버 연결 코드에 필요한 조건과 값을 채웁니다. 선택 심화에서는 문서 변경을 비교할 자료구조와 처리 순서를 직접 구성합니다.

🧭 과제 전체 지도

문제	완성할 기능	핵심 결과
필수 1	공개 자료 수집과 저장	원천 JSON, 수집 결과
필수 2	품질 점검과 문서 구성	채택, 격리, 제외 및 품질 기록
필수 3	문서 API	문서 등록, 조회, 입력, 응답 검증
필수 4	질문 API	모델 연결 + 실패 처리 + 요청 제한과 기록
선택 심화 1	문서 변경 내역	두 시점의 스냅샷 비교

필수 4의 세 기능은 모두 구현해야 합니다. 아래 기능명은 단계별 검사 단위이며 별도 문제 번호가 아닙니다. 뒤 단계가 미완성이어도 앞 기능부터 확인할 수 있습니다.

문제 1. 공개 자료를 모아 다시 읽을 수 있게 저장하기

문제 배경

릴리스 자료를 재사용하려면 페이지 종료, 상한, 실패를 구분하고 원천과 수집 상태를 함께 남겨야 합니다.

수행할 내용

1. GitHub 공식 API 설명에서 주소, 파라미터, 응답 필드, 호출 제한을 확인하고 `collect_releases` 의 페이지, 상태 검사, 다음 페이지 조건, 저장 값을 완성합니다.
2. GET `https://api.github.com/repos/astral-sh/uv/releases` 에 `page=1` 부터 요청합니다. 기본값은 `per_page=5`, `max_pages=2` 입니다. `response.links` 에 `next` 가 없으면 끝내고, 다음 페이지가 남은 채 상한에 도달하면 `limited=true` 로 남깁니다.
3. HTTP timeout, 상태, JSON 파싱, 최상위 배열을 확인합니다. 오류가 나면 재시도하거나 건너뛰지 않고 중단합니다. 실패 실행의 부분 원천이나 이전 성공 파일을 정제 성공 자료로 사용하지 않습니다.

구현 내용

구현 코드와 작성 메모, UTF-8 `raw_pages.json` 과 `collection_report.json`, 실행, 검사 결과. 원천 파일은 성공한 페이지의 원본 객체를 `[{"page": 1, "items": [...]}, ...]` 로 보관합니다. 보고서에는 mode, source, collected_at, successful_pages, raw_count, success, failed_page, limited, error를 남기며 실패 페이지와 오류가 없으면 `failed_page` 와 `error` 는 null입니다.

결과 확인

```
uv run --frozen python app.py run collect
uv run --frozen python app.py check collect
uv run --frozen python app.py run collect --live
```

- 1페이지에 next가 있고 2페이지에 없으면 2페이지 저장, `success=true`, `limited=false`.
- 2페이지에도 next가 남으면 2페이지 저장, `success=true`, `limited=true`.
- 1페이지 성공 후 2페이지가 503이면 첫 페이지만 저장, `success=false`, `failed_page=2`.

저장 파일을 다시 읽어 구조와 건수를 비교하세요. 최대 2페이지 수집은 전체 릴리스 이력의 수집을 뜻하지 않습니다. 수집 실행이 문서 정제까지 실행하지는 않습니다.

문제 2. 믿고 사용할 문서로 정리하기

문제 배경

API 응답에도 결측, 형식 오류, 중복이 있습니다. 사용할 자료와 품질 문제, 수집 범위 밖 자료를 구분해야 근거를 추적할 수 있습니다.

수행할 내용

`normalize_releases` 의 자료형, 빈 본문 검사, 제목 대체, 중복 판단과 문서 값을 다음 순서로 완성합니다.

1. 객체인지, ID가 boolean을 제외한 양의 정수인지, `draft`, `prerelease` 가 모두 boolean인지 확인합니다. 문자열 `false` 는 허용하지 않습니다.
2. `draft=true` 또는 `prerelease=true` 는 내용 검사 전에 제외합니다.
3. 제목, 본문, URL, 게시시각의 형식과 길이를 검사합니다. `name` 이 누락, null, 공백이면 유효한 `tag_name` 을 쓰고, 숫자 등 다른 자료형은 격리합니다. 본문은 정리 후 비어 있지 않은 문자열이어야 하며 제목으로 대신하지 않습니다.
4. **이미 채택한** ID만 중복으로 격리합니다. 잘못된 첫 행이 뒤의 정상 행을 막으면 안 됩니다.
5. 원본을 수정하지 않고 새 문서를 만듭니다. 제공 `clean_text` 로 제목, 본문을 정리하고 `validate_url` 이 반환한 URL을 사용합니다. `html_url` 은 API의 내부 `url` 필드와 다릅니다. 시간대가 있는 ISO8601 `published_at` 원문은 품질 기록에 보존하고 문서에는 별도의 수집시각을 넣습니다.

공통 문서 규격 — 정제 결과와 API는 다음 여섯 필드만 사용합니다. 문자열은 앞뒤 공백을 정리한 뒤 검사하며 숫자를 문자열로 자동 변환하지 않습니다. 상한 초과 값을 조용히 잘라 쓰지 않습니다.

필드	조건
`doc_id`	영문, 숫자, 하이픈, 밑줄 1~100자. 정책 `policy-*`, 수집 문서 `release-uv-{id}`
`title`	문자열 1~300자
`body`	문자열 1~20,000자
`source_url`	HTTP(S) URL 1~2,048자, host 필수, 내부 공백 금지
`collected_at`	UTC ISO8601 문자열
`category`	installation, access, incident, general, release 중 하나

제공 HTML 정리는 entity를 변환하고 `p`, `br`, `div`, `b`, `strong`, `em`, `code`, `span`, `li`, `ul`, `ol` 의 단순 태그를 공백으로 바꾼 뒤 연속 공백을 정리합니다. 단어가 붙지 않게 하고 `<T>` 같은 다른 텍스트를 임의로 지우지 않습니다. 생성한 `release-uv-{id}` 가 100자를 넘으면 `invalid_id` 로 격리합니다.

구현 내용

구현 코드와 작성 메모, `documents`, `quarantine`, `excluded`, `quality_report` 를 가진 반환 dict와 실행부가 같은 이름으로 저장한 JSON 파일. 격리, 제외 기록은 page, row, id, reason을 남기며 page/row는 1부터 셉니다. 품질 기록은 input_count, kept_count, quarantine_count, excluded_count 및 각 행의 page, row, id, published_at, status, reason을 보존합니다. 사유 코드와 상세 형식은 제공 자료 설명을 따릅니다.

결과 확인

```
uv run --frozen python app.py run documents
uv run --frozen python app.py check documents
uv run --frozen python app.py run documents --live
```

고정 입력은 **10건 = 채택 3건 + 격리 5건 + 제외 2건**이며 채택 ID는 `release-uv-101`, `release-uv-102`, `release-uv-109` 입니다. 빈 제목은 tag_name으로 보완하고, 같은 ID의 첫 행이 잘못됐어도 뒤 정상 행은 채택되며, 초안의 빈 본문은 격리가 아닌 제외인지 확인하세요. 같은 입력, 수집시각에는 같은 결과가 나와야 합니다. live 정제는 직전에 성공한 live 원천을 사용하며 네트워크를 호출하지 않습니다. 수집을 다시 했다면 같은 모드의 정제도 다시 실행합니다.

문제 3. 다른 프로그램이 사용할 문서 API 만들기

문제 배경

문서 조회, 등록을 다른 서비스가 사용할 수 있도록 입력, 응답 형식과 HTTP 상태를 일관되게 제공해야 합니다.

수행할 내용

`Document`, `QueryRequest`, `QueryReply` 의 예시 아래에 모델 필드 9개를 선언한 뒤 각 검증 함수의 `support.todo` 한 줄을 지웁니다. `find_document`, `register_document`, `response_models` 에 없는 ID, 중복 등록 조건과 응답 모델을 채웁니다. 제공된 공백, URL, 날짜, 중복 목록 검사 틀과 서버 연결을 활용하세요.

요청	정상 동작 / 오류
`GET /health`	200. status/mode/document_count, 현재 메모리 문서 수
`GET /documents`	200. Document 배열, 빈 목록도 정상
`GET /documents/{doc_id}`	200. Document / 없는 ID 404
`POST /documents`	201. 등록한 Document / 같은 ID 또는 URL 409, 형식 오류 422
`POST /query`	필수 4에서 200과 QueryReply 구현. 문서 API 단계에서는 501

문서 등록에는 앞의 여섯 필드를 모두 받습니다. 처음에는 정책 6개를 읽고 정상 fixture 정제 후 총 9개가 됩니다. 현재 모드의 수집 결과가 있으면 정제까지 마쳐야 서버를 시작할 수 있습니다. 실패 수집, 오래된 정제, 중복 자료는 제공 실행부가 확인합니다. POST 등록은 **메모리 전용**이며 재시작하면 파일을 다시 읽어 수동 등록 문서는 사라집니다. 이 동작을 `/docs` 에서도 확인하세요.

구현 내용

구현 코드와 조회 함수의 작성 메모, 등록, 조회, 중복, 형식 오류의 요청/응답, 직접 선택한 경계값 1개의 요청/응답. `/docs` 의 스키마도 요청, 응답 규격과 대조하세요.

결과 확인

```
uv run --frozen python app.py serve --stage document-api
uv run --frozen python app.py check document-api
```

서버는 터미널 1에서 실행하고 검사 등 다른 명령은 같은 폴더의 터미널 2에서 실행합니다. 브라우저의 `http://127.0.0.1:8000/docs` 에서 `Try it out > Execute` 로 health, 문서 목록을 확인하고, 과제 본문의 여섯 필드 예시로 등록 후 조회하세요. 같은 문서를 다시 등록하면 409, `manual/a` 처럼 잘못된 ID는 422여야 합니다. 누락, 자료형, 빈 값도 확인합니다. 빈 저장소 검사는 제공되므로 정책 파일을 지우지 않습니다.

종료는 `Ctrl+C` 입니다. 코드나 정제 파일을 바꿨다면 종료 후 재시작하세요. 8000번 포트가 사용 중이면 `uv run --frozen python app.py serve --stage document-api --port 8001` 로 실행하고 브라우저 포트도 맞춥니다. 서버는 내 컴퓨터의 단일 프로세스로 실행합니다.

문제 4. 질문 API에 LLM을 연결하고 실패 처리하기

문제 배경

모델의 HTTP 요청 성공만으로 답변을 신뢰할 수는 없습니다. 선택 문서에 근거한 응답을 검증하고 외부 실패, 요청 한도도 구분해야 합니다. **모델 연결, 실패 처리, 요청 제한과 기록을 모두 구현**합니다.

수행할 내용 1. 모델 연결

`build_prompt` 에서 질문과 선택 문서를 dict에 담고 `call_model_once` 에서 응답 JSON과 출처를 검사합니다. HTTP 요청, `await`, 모델 API 요청 형식은 제공합니다.

- 요청 `question` : 공백 정리 후 문자열 3~1,000자. `document_ids` : 필수, 서로 다른 DocumentId 1~3개. `session_id` : 기본 `demo`, 공백 정리 후 문자열 1~64자이며 대화 이력을 저장하지 않습니다.
- DocumentId는 `doc_id` 와 같은 규칙입니다. 추가 필드, 형식 오류는 422, 없는 ID는 **모델 호출 전 404**입니다. `GET /documents` 에서 선택한 문서의 실제 본문, 메타데이터만 전달합니다. 검색, 자동 문서 선택은 구현하지 않습니다.
- `POLICY_INSTRUCTIONS` 에는 기본 정책 지침이 들어 있습니다. 이전 프롬프트를 사용한다면 역할, 근거 확인 기준만 옮기고 문서 6개를 포함한 이전 `system_prompt.txt` 전체는 붙이지 않습니다.
- 답변 `answer` : 공백 정리 후 비어 있지 않은 문자열. `source_ids` : 중복 없는 DocumentId 배열이며 현재 선택 문서의 ID만 허용. `needs_human` : boolean이며 문자열 false와 숫자 0/1은 거부. `category` : installation, access, incident, general 중 하나.
- 출처가 비면 `needs_human=true` 여야 합니다. 담당자 확인이 필요한 정상 답변은 200이며 모델 장애를 이런 정상 답변으로 바꾸지 않습니다.

수행할 내용 2. 실패 처리

`query_with_retry` 의 재시도 조건 두 개와 공급자 429의 응답 상태를 채웁니다. `attempt` 는 첫 시도 0, 두 번째 1입니다. timeout, 5xx만 한 번 재시도하며 반복, 비동기 대기, HTTP timeout, 한 시도 전체 제한 시간은 제공됩니다.

상황	최종 상태 / 모델 호출 수
정상	200 / 1회
첫 timeout 또는 5xx 뒤 성공	200 / 2회
timeout 두 번	504 / 2회
5xx 두 번	502 / 2회
공급자 429	503 / 1회
다른 4xx, 연결, JSON, 스키마, 출처 오류	502 / 1회
재시도 후 다른 실패	마지막 실패 종류의 위 오류 / 2회
live 키 미설정	503 / 0회

오류는 제공된 짧은 설명으로 반환하며 원문 예외나 키를 노출하지 않습니다. 생성 요청의 재시도는 호출과 비용을 중복시킬 수 있습니다. 실패 검사는 고정 대역으로 수행하고 실제 모델을 고의로 실패시키지 않습니다.

수행할 내용 3. 요청 제한과 기록

`admit_query` 에 시간 구간 갱신, 한도 초과, 요청 수 증가를, `request_logging` 에 로그 필드를 작성합니다. 서버 연결과 미들웨어의 `await call_next` 는 제공합니다.

- `POST /query` 만 **단일 프로세스 전체 30회/60초**로 제한하며 모든 호출자가 공유합니다. 형식과 문서 ID를 먼저 확인하여 422/404는 횟수에서 제외합니다.
- `admit_query` 는 허용 시 0, 초과 시 남은 초를 올림한 양의 정수를 반환합니다. 초과 응답은 429와 `Retry-After` 이며 모델을 부르지 않습니다.
- 구간 종료 시 횟수를 초기화하고 상태 갱신은 await 전에 마칩니다. 모델 재시도는 추가 요청으로 세지 않습니다.

- 로그는 method, 경로 양식, status, mode, elapsed_ms만 남깁니다. 경로는 `support.safe_route(request)` 를 사용하고 응답에는 `X-DevDesk-Mode` 를 설정합니다. 질문, 문서 본문, 인증 헤더, 원문 예외는 기록하지 않습니다.

구현 내용

세 기능의 코드, 재시도 함수의 작성 메모, fixture 검사 결과, 실제 모델 응답과 선택 문서 대조 결과, 서버 요청, 응답, 모드 및 허용 필드로만 구성된 로그.

결과 확인

```
uv run --frozen python app.py run query
uv run --frozen python app.py check query
uv run --frozen python app.py run retry
uv run --frozen python app.py check retry
uv run --frozen python app.py serve
uv run --frozen python app.py check observe
uv run --frozen python app.py run query --live
uv run --frozen python app.py serve --live
```

각 서버 명령은 필요할 때 하나씩 실행합니다. `run query` 는 모델 1회 연결 확인이며, `serve --live` 에서는 `/query` 요청 시 실제 모델을 호출합니다. 포트가 겹치면 fixture 서버는 `uv run --frozen python app.py serve --port 8001` , live 서버는 `uv run --frozen python app.py serve --live --port 8001` 을 사용하고 브라우저 포트도 맞춥니다.

정상 응답 후 잘못된 JSON, 자료형, 선택 밖 출처를 거부하는지 확인합니다. 재시도 검사에서는 503 뒤 성공 시 2회, 401 시 1회 호출이어야 합니다. 제한 검사 `limit=2` , `window=10` 에서는 0초, 1초 허용, 2초 거부, 10초 새 구간이며 차단 시 모델 호출은 0회입니다. 고정 지연 요청 3개의 대기가 겹치는지도 제공 검사로 확인합니다. 이는 운영 규모 부하나 분산 환경 검증은 아닙니다.

fixture 검사 후 정상 요청으로 실제 모델을 확인하고 문장을 선택 문서와 직접 대조하세요. 실제 실패, 한도 소진 실험은 필요하지 않습니다. `check all` 통과가 실제 연결이나 답변 사실성까지 확인했다는 뜻은 아닙니다. 외부 연결이 막히면 오류와 확인 범위를 구분해 운영자에게 문의하며 live 설정 오류를 fixture 성공으로 대신 기록하지 않습니다.

🚀 선택 심화 1. 두 시점의 문서 변경 내역 제공하기

문제 배경

같은 범위의 문서 스냅샷을 비교하여 추가, 변경, 유지, 부재를 구분합니다. 제한된 실제 GitHub 두 페이지를 완전한 이력으로 간주하지 않습니다.

수행할 내용

`compare_snapshots(before, after)` 의 자료구조와 처리 순서를 직접 구성합니다. 전체 스냅샷 2개와 `GET /document-changes` 연결은 제공합니다.

- ID별 `added` , `changed` , `unchanged` , `missing` 을 구분합니다.
- `title` , `body` , `source_url` , `category` 의 변경 필드를 `changed_fields` 에 담고 added/missing은 빈 배열로 둡니다. 수집시각만 바뀌면 unchanged입니다.
- 입력을 수정하지 않고 결과를 doc_id 순서로 정렬합니다. 빈 입력, 입력 순서 변경, 한 문서의 여러 필드 변경도 처리합니다. missing은 비교 자료에 없다는 뜻이며 실제 삭제를 뜻하지 않습니다.

구현 내용 및 확인

구현했다면 아래 명령을 실행해 문서 변경 비교와 검사 결과를 확인합니다. 필수 검사와 분리되어 있으며 미구현이어도 필수 서버와 검사는 실행됩니다.

```
uv run --frozen python app.py challenge
uv run --frozen python app.py check challenge
```

✅ 제출 전 체크리스트

☐ 필수 1~4를 구현했고 필수 4의 세 기능을 모두 연결했습니다.
☐ 수집, 정제, 조회, 재시도 함수의 `작성 메모` 에 직접 판단한 이유를 1~2줄씩 남겼습니다.
☐ 단계별 검사 후 `uv run --frozen python app.py check all` 을 실행하고 `artifacts/fixture/checks-기능명.json` 을 확인했습니다. 통과, 실패 수는 검사 항목 수이며 문제 수가 아님을 이해했습니다.
☐ 원천, 정제 건수와 채택/격리/제외 기록을 대조했고, 원본 불변과 같은 입력의 재실행 결과를 확인했습니다.
☐ 실제 수집 → 같은 모드의 정제 → 서버/모델 확인을 진행하고 fixture와 live 결과를 구분했습니다.
☐ `/docs` , 등록, 조회, 중복, 오류, 직접 선택한 경계값의 요청/응답 및 재시작 동작을 확인했습니다.
☐ 실제 답변을 선택한 근거 문서와 대조하고 실패, 제한, 회복은 제공 대역으로 검사했습니다.
☐ `app.py` 와 `AI_USAGE.md` 두 파일만 제출하며 제공 파일, 데이터, 실행 결과와 비밀값을 제외했습니다.

▶ LangChain 과제